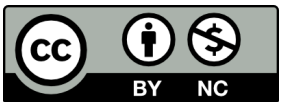
Business 4720

The Linux/Unix Command Line

Joerg Evermann



1 The Unix/Linux Command Line (also for Mac and Windows users)

This tutorial provides a very brief introduction to the Ubuntu command line ("terminal"). The command line, also called a "shell" is by default the "bash" shell (Bourne-again shell; a pun on the earlier Bourne shell).

Tip

The default shell in the MacOS terminal is the "zsh" and behaves slightly differently from the bash shell. You can work with a "bash" shell by typing the `bash` command in a MacOS terminal.

Tip

When using WSL (Windows Subsystem for Linux) and the default Ubuntu installation, the default shell in the WSL terminal for Ubuntu is also the "bash" shell.

Resources

The following are further beginner-level tutorials on using the command line on Ubuntu (or any other Linux distribution):

- Ubuntu command line for beginners
- Linux command line primer
- Getting started with Linux

Bash will show you a *command prompt* that indicates your username, the name of the computer, and your current working directory followed by the dollar sign "$". In the following example, the username is "busi4720", the name of the computer is also "busi4720vm" and your current working directory is the home directory, indicated by "~":

You can print the working directory by typing the `pwd` command and then pressing the Return or Enter key:

bash

```
busi4720@busi4720vm:~$ pwd
/home/busi4720
```

You can make a folder/directory with the `mkdir` command (in your current working directory):

bash

```
busi4720@busi4720vm:~$ mkdir someFolder
```

You can change the working directory to the folder you have just created with the `cd` command. Note how the Bash command prompt indicates your new working directory.

bash
```
busi4720@busi4720vm:~$ cd someFolder
busi4720@busi4720vm:~/someFolder$ cd ..
busi4720@busi4720vm:~$ cd ~
```

The following special characters can be used when specifying folders/directories and paths:

~	User home directory
.	Current directory
..	Upwards in the directory tree
/	Root of directory tree

Tip

Here are some tips that make working with the shell a lot easier:

- Autocompletion of file names is available with the "Tab" key. When multiple file names exist that match what you have entered so far, you can enter further characters of a file name to disambiguate and press the Tab key again for further autocompletion.
- You can recall earlier commands with the "Up Arrow" key. By default, the shell stores the last 1000 commands.
- You can search earlier commands with the "Ctrl-R" key. You are then prompted to search by typing in characters to find commands. The shell finds the most recent command that contains the characters you entered. At any time you can press Ctrl-R again to find earlier matches to your command search.
- Because the usual keys Ctrl-X, Ctrl-C, Ctrl-V for cutting, copying, and pasting text have different functions in the shell, you can cut, copy, and paste with the Ctrl-Shift-X, Ctrl-Shift-C, and Ctrl-Shift-V keys.

You can list folder/directory contents using the `ls` command. The option `-l` for the command indicates that you would like to see long results.

bash
```
busi4720@busi4720vm:~$ ls -l ~/Applications
total 8
drwxrwxr-x 7 busi4720 busi4720 4096 Nov  8 12:05 arcadedb-23.10.1
drwxr-xr-x 8 busi4720 busi4720 4096 Nov  7 11:45 pycharm-community-2
```

The results show the total size in kB (kilobytes), and a list of entries:

- Type of entry (”d” = directory)
- Permissions for owner of the file (”rwx”), users in the same user group as the owner (”r-x”) and other users (”r-x”): `r` indicates read access, `w` indicates write access, `x` indicates permission to run the application or enter/view a directory (with `cd` or `ls`), and a – indicates the lack of the corresponding permission.
- Names of owner and groups (”busi4720”)
- Size (in bytes)
- Last modification date and time
- File or directory name

Printing a string of text uses the `echo` command:

```bash
$ echo "To be or not to be"
To be or not to be
```

Redirect the output of the `echo` command to a file using the *redirect* symbol ”>”. You can redirect the output of any command this way. Use » to redirect and append, instead of overwriting a file.

```bash
$ echo "To be or not to be" > someFile.txt
$ ls -l someFile.txt
-rw-rw-r-- 1 busi4720 busi4720 19 Nov  8 14:50 someFile.txt
```

Print contents of a file (”concatenate”) using the `cat` command. You can concatenate multiple files by specifying them all (this is why the command is called ”concatenate”):

```bash
$ cat someFile.txt
To be or not to be
```

If you would like to see the contents of a file page by page or line by line, use the `less` command (a pun on ”less is more” and the earlier ”more” command that did the same). You will be shown the contents and can navigate up and down with the usual arrow keys.

```bash
$ less someFile.txt
```

You can copy a file using the `cp` command:

```bash
$ cp someFile.txt someCopy.txt
```

You can move a file to a new location (folder/directory) using the `mv` command:

```bash
$ mv someCopy.txt ~/someFolder
```

Renaming is moving. When you want to rename a file, move it to a new file name:

```bash
$ mv someFile.txt newName.txt
```

You can remove (delete) a file using the `rm` command:

```bash
$ rm someFolder/someFile.txt
```

Remove a directory recursively (i.e. remove all its contents first):

```bash
$ rm -r ~/someFolder
```

Important

Use the `rm` command very carefully! You could inadvertently delete all your files. The shell will delete files and folders immediately and irrevocably. There is no "undoing" this.

View the command line history with the `history` commands. Remember that you can redirect this output to a file if you wish or use a pipe to pipe it into the less command (see below).

```bash
$ history
    1  echo "To be or not to be"
    2  echo "To be or not to be" > someFile.txt
    3  ls -l someFile.txt
    4  less someFile.txt
    5  cat someFile.txt
...
```

Management of file permissions is done using the `chmod` command. You can gran and revoke read, write, and execute permissions for yourself, your group members, and other users. Remove write permission for yourself by using the `-w` option and specifying the filename:

```bash
$ chmod -w newName.txt
```

Add write permissions using the `+w` option:

```bash
$ chmod +w newName.txt
```

Add execute permissions using the `+x` option:

```bash
$ chmod +x newName.txt
```

If you are stuck on how to use a command or wish to see all its options and capabilities, you can get the manual for a command using the `man` command:

```bash
$ man ls
```

If you can't quite remember which command to use, you can search for commands using keywords with the `apropos` command:

```bash
busi4720@busi4720vm:~$ apropos python
keyring (1)          - Python-Keyring command-line utility
pdb3 (1)             - the Python debugger
pdb3.10 (1)          - the Python debugger
pip (1)              - A tool for installing and managing Python p...
pip3 (1)             - A tool for installing and managing Python p...
py3compile (1)       - byte compile Python 3 source files
py3versions (1)      - print python3 version information
pydoc3 (1)           - the Python documentation tool
pydoc3.10 (1)        - the Python documentation tool
pygettext3 (1)       - Python equivalent of xgettext(1)
pygettext3.10 (1)    - Python equivalent of xgettext(1)
python (1)           - an interpreted, interactive, object-oriente...
python3.10-config (1) - output build options for python C/C++ exte...
python3 (1)          - an interpreted, interactive, object-oriente...
...
```

You can see a list of all processes currently running using the `ps` command. The results show the process identifier (PID), the console from which you started the process, the computing time it has consumed, and the command that was used to start the process. Add the `a` and `x` option to see *all* the processes running on your computer, not just the processes you have started.

```bash
$ ps ax
    PID TTY          TIME CMD
   3024 pts/0    00:00:00 bash
   3151 pts/0    00:00:00 ps
```

The `grep` command is useful to find something in a file or input stream. Use it as in the following example in a pipe:

```bash
$ cat newName.txt | grep be
$ ls -l | grep .txt
$ history | grep .txt
```

Note: The vertical bar is called a *"pipe"*, it pipes the output of one command as input into the next one

The following are a set of connected exercises to help you practice your command line skills. Do them in the order listed.

Hands-On Exercise

1. Navigation and Listing
   (a) Open the terminal and use the `pwd` command to print the current working directory.
   (b) Use `ls` to list the contents of the current directory.
   (c) Create a new directory named "Exercise1" using `mkdir`.
   (d) Navigate into the "Exercise1" directory using `cd`.
2. File Manipulation
   (e) Create a new file named "file1.txt" inside the "Exercise1" directory using `touch`.
   (f) Use `cat` to display the contents of "file1.txt".
   (g) Append the text "Hello, Bash!" to "file1.txt" using `echo` and ».
   (h) Display the updated contents of "file1.txt" using `cat`.
3. Removing and Renaming
   (i) Remove "file1.txt" using the `rm` command.
   (j) Create a copy of the "Exercise1" directory named "Exercise1_backup" using `cp -r`.
   (k) Remove the original "Exercise1" directory using `rm -r`.
4. Directory Manipulation
   (l) Recreate the "Exercise1" directory.
   (m) Create three subdirectories inside "Exercise1" named "Subdir1", "Subdir2", and "Subdir3" using `mkdir`.
   (n) List the contents of "Exercise1" to verify the creation of subdirectories.
5. Searching and Filtering
   (o) Create a file named "keywords.txt" inside "Exercise1" and add some random text.
   (p) Use `grep` to search for a specific word (e.g., "Bash") in "keywords.txt".
   (q) Create a new file named "filtered.txt" and use `grep` to filter lines containing the word you searched for in "keywords.txt".
6. Process Management
   (r) Use `ps` to display information about the current processes running on your system.
   (s) Use `ps aux | grep bash` to filter and display information about Bash processes.
7. Cleanup
   (t) Remove the entire "Exercise1" directory and its contents using `rm -r`.
   (u) Confirm that the "Exercise1" directory no longer exists by listing the contents of the current directory.